



Python – Ablaufkontrolle

Stephan Karrer

Autor: Stephan Karrer

Bedingte Anweisungen

```
if a==3:
    print("a ist 3")
elif a==4:
    print("a ist 4")
elif a==4:
    print("a ist 5")
else:
    print("weder 3 noch 4 noch 5")
```

- elif und else sind optional
- Die Einrückung bestimmt den Block, Klammern sind nicht nötig

Was ist False oder True

```
# auch leere Sequenzen liefern False
liste = []

if liste:
    print("Liste ist nicht leer")
else:
    print("Liste ist leer")
```

- Folgendes liefert in Python False:
 - numerische Null-Werte(0, 0L, 0.0, 0.0+0.0j)
 - der boolsche Wert False
 - leere Sequenzen
 - leere Dictionaries
 - der spezielle Wert None
- Alles andere liefert True

Die leere Anweisung: pass

```
if x == 1:
    pass
else:
    print("x hat Wert ungleich 1")
```

- Um Syntax-Fehler während der Programm-Entwicklung zu vermeiden, kann eine leere Anweisung formuliert werden.

Zuweisung innerhalb Bedingung (assignment expression)

```
geheimnis = 122
while (versuch := int(input("Raten Sie: "))) != geheimnis:
    if versuch == 0:
        print("Das Spiel wird beendet")
        break
    elif versuch < geheimnis:
        print("Zu klein")
    elif versuch > geheimnis:
        print("Zu groß")
    else:
        print("Sie haben es geschafft!")
```

- Innerhalb einer Bedingung kann eine Wertzuweisung erfolgen
 - Dies verhindert eine mehrfache Referenzierung desselben Werts

Bedingte Zuweisung (Ternäres if)

```
if a > b:
```

```
    max=a
```

```
else:
```

```
    max=b
```

```
# kann auch geschrieben werden als
```

```
max = a if (a > b) else b
```

```
# und direkt in Ausdrücken verwendet werden
```

```
a = 17
```

```
b = 50
```

```
result = (21 if a < b else 7) * 2
```

Bedingte Schleifen

while (*Bedingung*) { *Anweisungen* } //abweisend

```
i = 1
summe = 0
while i < 101:
    summe += i
    i += 1
else:
    print("jetzt ist i > 100")
print("Summe der Zahlen von 1 bis 100:", summe)
```

- Bis auf den optionalen ELSE-Zweig übliches Konstrukt in Programmiersprachen
- Selbstverständlich können Schleifen geschachtelt werden

Break und Continue

```
while ... :  
    ...  
    if ... :  
        break;  
    ...  
    while ... :  
        ...  
        if ...:  
            continue ;  
        ...
```

- Mit break kann die jeweilige Schleife verlassen werden, wobei dann kein eventuell vorhandener else-Zweig ausgeführt wird
- Mit continue kann der jeweilige Durchlauf abgebrochen werden, und der nächste Durchlauf gestartet werden
- Es gibt aber keine Sprungmarken für geschachtelte Schleifen

for-Schleife

```
for <target> in <iterable>:  
    Anweisungen  
[else:  
    Anweisungen]
```

```
summe = 0  
for i in range(101):  
    summe += i  
else:  
    print("jetzt ist i > 100")  
print("Summe der Zahlen von 1 bis 100:", summe)  
  
# oder noch einfacher  
print("Summe der Zahlen von 1 bis 100:", sum(range(1,101)))
```

- Schleife basiert intern darauf, dass die Wertemenge einen Iterator liefert
- Es ist in dieser Form nur Lesen ohne Schrittkontrolle möglich

Spezialform der for-Schleife bei Listen (List Comprehension)

```
lst = [1,2,3,4,5,6,7,8,9]
lst_neu = [x**2 for x in lst if x%2 == 0]
print (lst_neu)      # [4, 16, 36, 64]

tuple = (1,2,3,4,5,6,7,8,9)
neu = [x**2 for x in tuple if x%2 == 0]
print (type(neu))    # <class 'list'>

eu = [x**2 for x in range(10) if x%2 == 0]
print (type(neu))    # <class 'list'>
```

- Statt expliziter for-Schleife kann auf einfachere Art eine neue Liste erzeugt werden
- Ist auch für andre iterierbare Konstrukte nutzbar und liefert stets Liste

Pattern Matching mit Literalen

```
x = int(input("Zahl: "))
match x:
    case 5:
        result = 5**2
        print("Quadrat von 5:", result)
    case 3 | 4:          # Oder
        print("3 oder 4")
    case _:
        print("default")
```

- Mittels `match ... case` wird eine Mehrfachverzweigung realisiert.
- Dabei wird der Ausdruck hinter `match` nacheinander gegen die jeweiligen Muster getestet, die im einfachsten Fall Literale sind, bis eines passt.
- Mehrere Muster können mit „Oder“ kombiniert werden.
- Sofern kein Muster getroffen wird, passiert nichts.
- Mittels des Wildcard-Bezeichners `_` kann ein Default-Zweig realisiert werden. Dieser muss am Ende stehen, da sonst die anderen Zweige nicht erreicht werden können.
- Literale werden mit dem `==`-Operator geprüft, die Konstanten `True`, `False` und `None` mit dem `is`-Operator.

Pattern Matching – Default-Zweig mit Variable

```
x = 2
default = 0
print(default)      # 0

match x:
    case 5:
        result = 5**2
        print("Quadrat von 5:", result)
    case 3 | 4:
        print("3 oder 4")
    case default:
        print(default) # 2

print(default) # 2
```

- Benutzt der Default-Zweig einen beliebigen Bezeichner statt des Wildcard-Zeichens, so wird dieser als Variable interpretiert und auf den Eingangswert gesetzt.
- Vorsicht: Eine eventuell vorhandene Variable wird neu gesetzt.

Pattern Matching – Bindung von Variablen

```
l = input("Bitte zwei Ganzzahlen eingeben: ").split()
point = (int(l[0]), int(l[1]))      # point is an (x, y) tuple

match point:
    case (0, 0):
        print("Origin")
    case (0, y):                    # Variable y wird gebunden
        print(f"Y={y}")
    case (x, 0):                    # Variable x wird gebunden
        print(f"X={x}")
    case (x, y):                    # Variablen x und y werden gebunden
        print(f"X={x}, Y={y}")
```

- Bei strukturierten Datentypen wird der jeweilige Wert an die Variable gebunden